Coforge

Sales Data ETL Pipeline

Python + Pandas

Data Cleaning, Validation, Reporting and Visual Analysis

From raw sales records to clean, accurate business reports

Chittamuru Sai Charith · 85010903

Abhideep Bonigala · 85010858

Akkinapalli Sai Pranav · 85010865

Galla Tagore Sri Sai · 85010916

Raw sales data needs cleaning before it can be trusted

Problem

Raw data can hold duplicates, missing data, invalid values and inconsistent dates.

Solution

Clean the data, calculate sales metrics, and generate reports and charts.

01

sales.csv

Raw sales records

02

Clean Data

Validated records

03

Insights

Sales metrics

04

Reports & Charts

Business results

Coforge

Project Objectives



Read and inspect sales.csv



Identify data-quality issues



Clean and validate records



Calculate sales metrics



Generate business reports



Export reusable CSV files and charts

Coforge

Technologies Used

Technology	Version	Why it is used
Python	3.11.4	Core language
Pandas	2.2.3	Load, clean and group data
Matplotlib	3.9.2	Create charts
Pytest	8.4.2	Automated tests
CSV files	Not applicable	Input and report format

Coforge

Dataset Overview

Each row represents one sales transaction.

123

Raw records

8

Columns

Columns in sales.csv	
order_id	order_date
customer	product
category	quantity
price	city

Coforge

Data Quality Issues Found

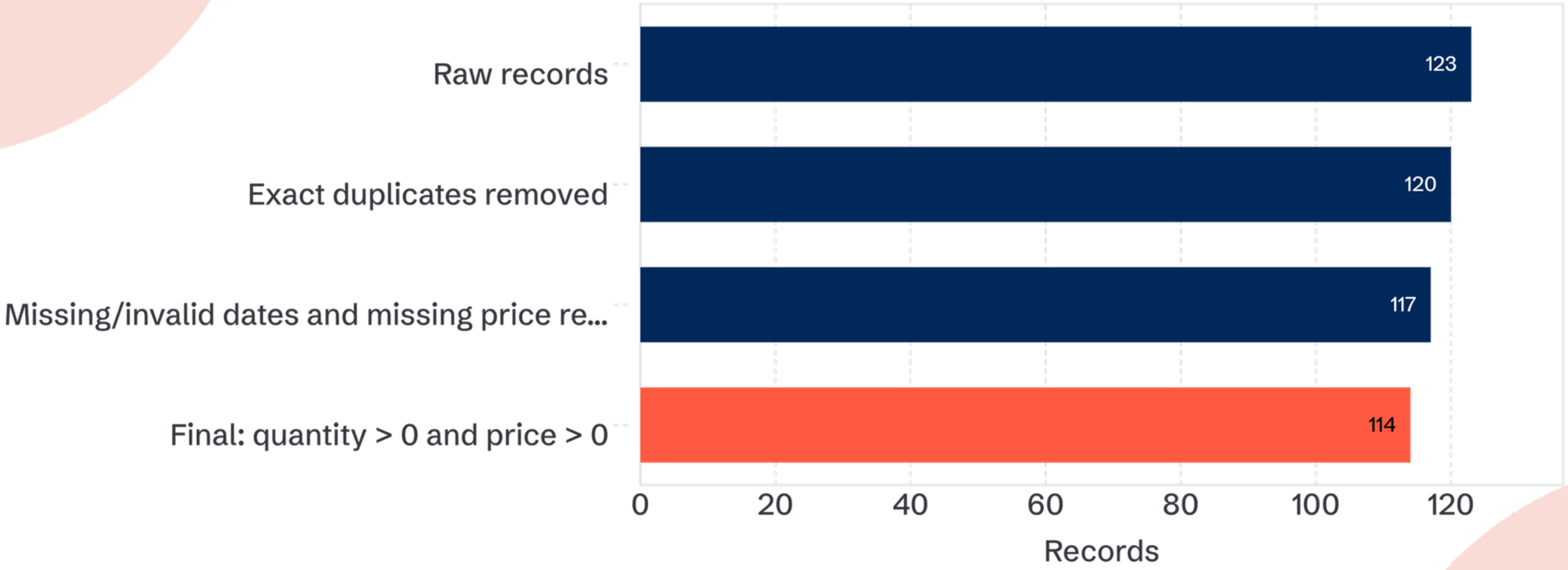
Issue	Count	Action
Exact duplicate rows	3	Removed
Missing customer	1	Replaced with "Unknown"
Missing city	1	Replaced with "Unknown"
Missing or invalid dates	2	Removed
Missing price	1	Removed
Invalid quantity: one zero, one negative	2	Removed
Invalid price: negative	1	Removed

Customer and city are kept as "Unknown"; invalid records are removed.

Coforge

123 raw records become 114 clean records

Cleaning stage



Each stage reduces the dataset to records that pass the cleaning rules.

Coforge

Key Transformation Logic

```
cleaned_data = data.drop_duplicates().copy()
cleaned_data["customer"] = cleaned_data["customer"].fillna("Unknown")
cleaned_data["city"] = cleaned_data["city"].fillna("Unknown")
cleaned_data["order_date"] = pd.to_datetime(
    cleaned_data["order_date"], errors="coerce", format="mixed")
cleaned_data = cleaned_data.dropna(subset=["order_date", "price"])
cleaned_data = cleaned_data[
    (cleaned_data["quantity"] > 0) & (cleaned_data["price"] > 0)]
```

```
data["sales_amount"] = data["quantity"] * data["price"]
```

Example: quantity 1 × price 650 = sales_amount 650

Coforge

Project Architecture and Code Structure

1. main.py
2. data_loader.py
3. data_validator.py
4. transformations.py
5. reports.py
6. charts.py
7. output/ and logs/

Project folders

- src/
- tests/
- output/
- logs/
- docs/

The processing flow runs from main.py to output/ and logs/.

Coforge

How Pandas Creates Sales Reports

Example from src/reports.py

```
product_sales = data.groupby(
    "product", as_index=False
)["sales_amount"].sum()

top_products = product_sales.sort_values(
    "sales_amount", ascending=False
).head(5)
```

1. **`groupby("product")`**
   Groups all rows for the same product.
2. **`sum()`**
   Adds the sales_amount for each product.
3. **`sort_values(..., ascending=False)`**
   Places the highest sales amounts first.
4. **`head(5)`**
   Selects the top five products.

> The same **groupby + sum** approach is used for customers, cities, categories, and monthly sales.

Overall Business Results

Total Sales

229,905.00

Average Order Value

2,016.71

Clean Records

114

Rows Removed

9

Coforge

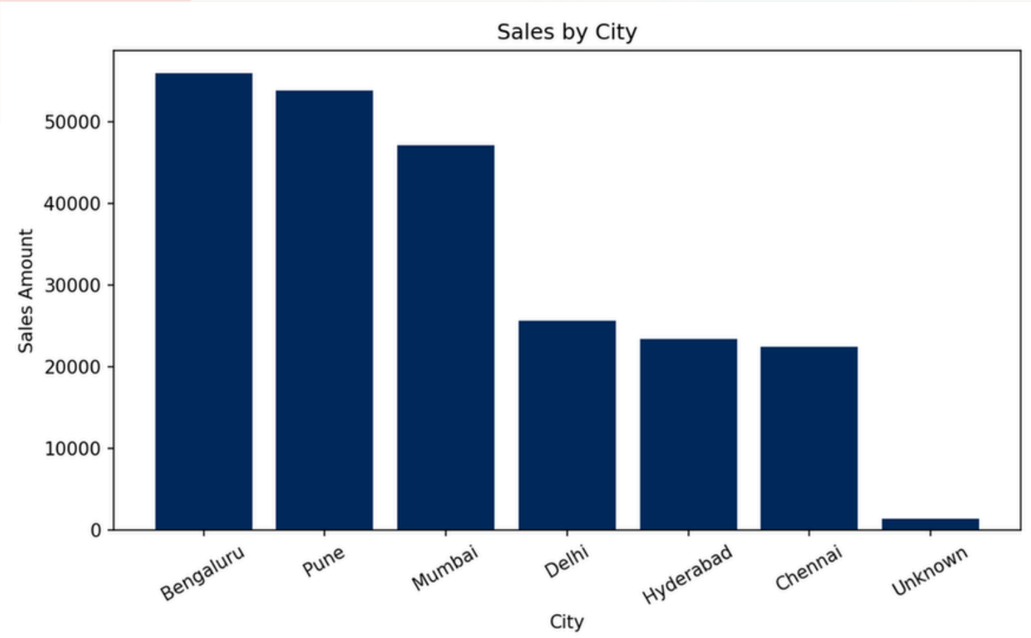
Top Products and Customers

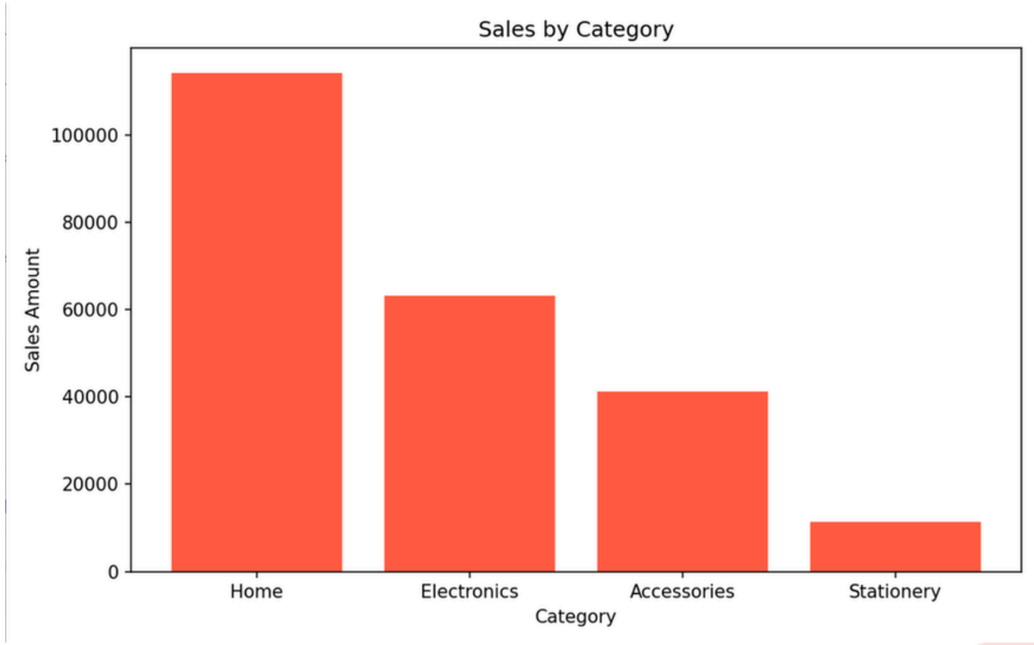
Top 5 Products		
1	Desk Lamp	76,500.00
2	Bluetooth Speaker	49,000.00
3	Phone Stand	27,650.00
4	Water Bottle	19,680.00
5	Coffee Mug	18,000.00

Top 5 Customers		
1	Arjun Das	27,980.00
2	Rohan Mehta	27,980.00
3	Neha Verma	27,230.00
4	Vikram Rao	26,615.00
5	Kabir Khan	26,480.00

Coforge

Sales by City and Category

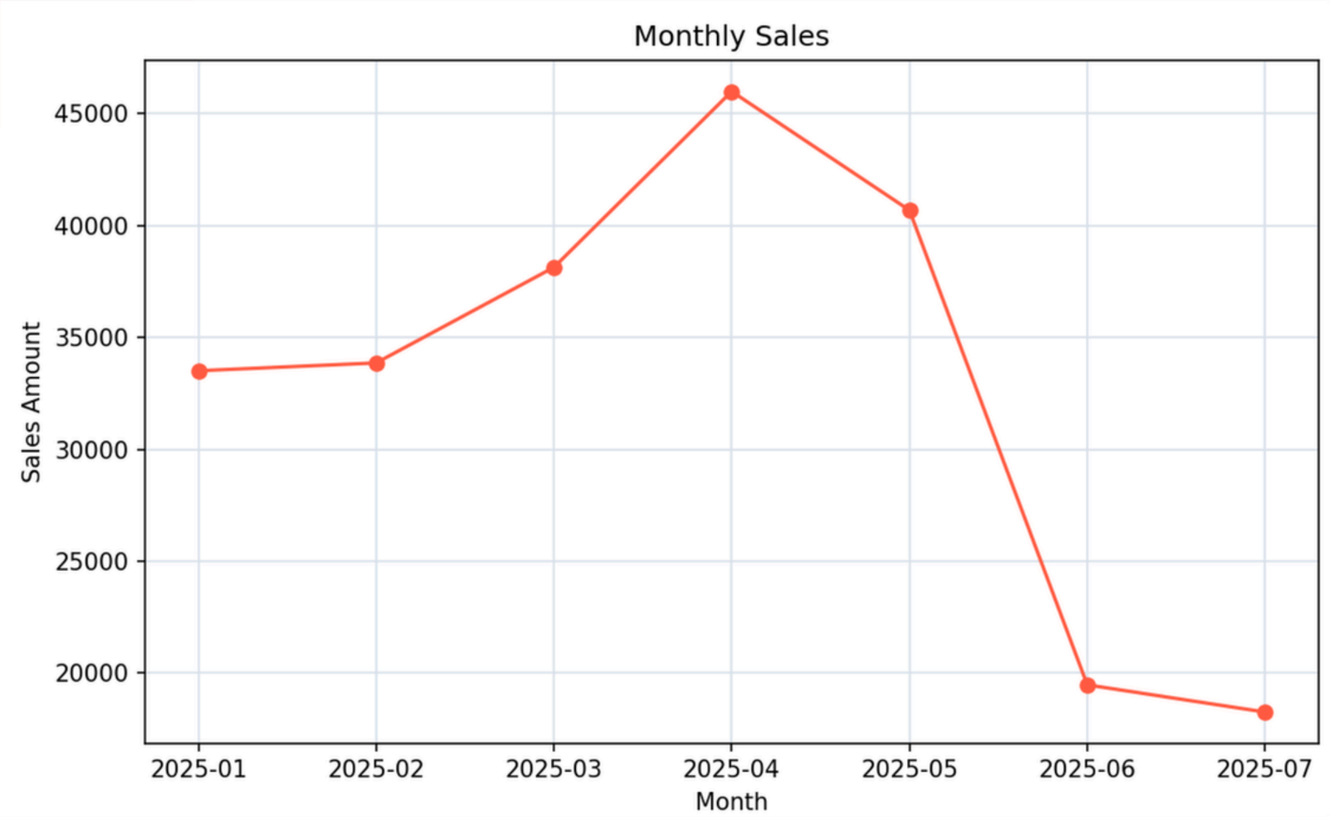




Bengaluru is the highest city: 55,960.00 · Home is the highest category: 114,180.00 · Missing cities are reported as Unknown, not ignored.

Coforge

Monthly Sales Trend



Highest monthly sales

April 2025 — 46,000.00

order_date was converted into YYYY-MM for monthly grouping.

Coforge

Automated Testing Example

Example from tests/test_transformations.py

```python
def test_add_sales_amount():
    test_data = pd.DataFrame(
        {"quantity": [2], "price": [150]}
    )

    result = add_sales_amount(test_data)

    assert result["sales_amount"].iloc[0] == 300
```

Input

Quantity = 2 · Price = 150

Expected result

2 × 150 = 300

Assertion

The test passes only when sales_amount equals 300.

Run command

```
py -m pytest
```



Current test result

4 passed

Coforge

Testing and Quality Assurance

4

Tests Passed

py -m pytest

Data cleaning test

Missing/invalid date test

sales_amount calculation test

Product-sales aggregation test

Coforge

Main Project Workflow — Part 1

main.py: Project Start, Loading, Validation and Audit Report

main.py

```python
logger = setup_logger()
logger.info("Sales Data Analysis project started")

# Load and check the raw sales data.
sales_data = load_sales_data(INPUT_FILE, logger)

if sales_data is None:
    logger.error("Project stopped because the input data could not be loaded")
    raise SystemExit

check_data_quality(sales_data)

# Save the audit report before cleaning the data.
data_quality_summary = create_data_quality_summary(sales_data)
data_quality_summary.to_csv(
    f"{OUTPUT_FOLDER}/data_quality_summary.csv", index=False
)
print(f"Data-quality audit saved: {OUTPUT_FOLDER}/data_quality_summary.csv")
logger.info("Data-quality audit report exported")
```

1. Starts the project logger.
2. Loads sales.csv and safely stops if input data cannot be loaded.
3. Checks data quality and creates the data-quality audit report.

Configuration Settings

config.py

```
INPUT_FILE = "sales.csv"
OUTPUT_FOLDER = "output"
LOG_FOLDER = "logs"
```


Coforge

Main Project Workflow — Part 1 (continued)

main.py: Cleaning, Sales Amount, Outlier Report and Totals

main.py

```
cleaned_sales_data = clean_sales_data(sales_data)
print("\nClean rows ready for analysis:", len(cleaned_sales_data))
logger.info("Data cleaning completed: %s clean rows", len(cleaned_sales_data))

# Add the sales value for each clean record.
cleaned_sales_data = add_sales_amount(cleaned_sales_data)

outlier_report = find_outliers(cleaned_sales_data)
outlier_report.to_csv(f"{OUTPUT_FOLDER}/outlier_report.csv", index=False)
print(f"Outlier report saved: {OUTPUT_FOLDER}/outlier_report.csv")
logger.info("Outlier report exported: %s rows", len(outlier_report))

total_sales = cleaned_sales_data["sales_amount"].sum()
average_order_value = cleaned_sales_data["sales_amount"].mean()
```

1. Cleans the raw data and logs the number of clean rows.
2. Adds sales_amount and exports the IQR outlier report.
3. Calculates total sales and average order value.

Main Project Workflow — Part 2

main.py: Business Results and Grouped Sales Reports

main.py

```
print("Total sales:", round(total_sales, 2))
print("Average order value:", round(average_order_value, 2))

top_products, top_customers, city_sales, category_sales, monthly_sales = create_reports(
    cleaned_sales_data
)

# Display the main analysis results.
print("\n--- Top 5 products by sales ---")
print(top_products)
print("\n--- Top 5 customers by sales ---")
print(top_customers)
print("\n--- Sales by city ---")
print(city_sales)
print("\n--- Sales by category ---")
print(category_sales)
print("\n--- Monthly sales ---")
print(monthly_sales)
```

1. Displays business results including total sales and average order value.
2. Creates grouped sales reports with create_reports().
3. Prints top products, top customers, city, category and monthly sales.

Coforge

Main Project Workflow — Part 2 (continued)

main.py: Report Export and Charts

main.py

```
export_reports(
    total_sales,
    average_order_value,
    top_products,
    top_customers,
    city_sales,
    monthly_sales,
)
print("\nCSV report files saved in the output folder.")
logger.info("Sales report CSV files exported")

create_charts(city_sales, category_sales, monthly_sales)
logger.info("Charts created and saved successfully")
```

1. Exports the grouped sales reports as CSV files.
2. Generates city, category, and monthly sales charts.
3. Logs each completed step to logs/app.log.

Loading Sales Data Safely — Part 1

src/data_loader.py · Required columns and safe file reading

src/data_loader.py

```python
def load_sales_data(file_name, logger):
    """Read and display a basic overview of the sales data."""
    required_columns = [
        "order_id",
        "order_date",
        "customer",
        "product",
        "category",
        "quantity",
        "price",
        "city",
    ]

    try:
        data = pd.read_csv(file_name)
    except FileNotFoundError:
        logger.error("Input file not found: %s", file_name)
        print(f"Error: The input file '{file_name}' was not found.")
        return None
    except Exception as error:
        logger.error("Could not read %s: %s", file_name, error)
        print(f"Error: Could not read '{file_name}'.")
        return None
```

1. Reads sales.csv using Pandas.
2. Lists the eight columns the project requires.
3. Shows clear errors when the input file cannot be read.

Coforge

Loading Sales Data Safely — Part 2

src/data_loader.py · Column check and data overview

src/data_loader.py

```
    missing_columns = [
        column for column in required_columns if column not in data.columns
    ]
    if missing_columns:
        logger.error("Required columns missing: %s", ", ".join(missing_columns))
        print("Error: Required columns are missing:", ", ".join(missing_columns))
        return None

    logger.info("Sales data loaded successfully: %s rows", len(data))

    print("\n--- First 5 rows ---")
    print(data.head())
    print("\n--- Dataset information ---")
    data.info()
    print("\n--- Missing values per column ---")
    print(data.isnull().sum())
    print("\n--- Duplicate rows ---")
    print(data.duplicated().sum())
```

1. Checks that required columns are present.
2. Logs how many rows were loaded successfully.
3. Prints the first rows, data types, missing values and duplicates.

Coforge

Data Validation and Quality Audit — Part 1

src/data_validator.py · Date conversion and validation checks

src/data_validator.py

```python
def get_converted_dates(data):
    """Convert dates so invalid values become missing date values."""
    return pd.to_datetime(data["order_date"], errors="coerce", format="mixed")


def check_data_quality(data):
    """Show the data-quality problems that need to be cleaned."""
    print("\n--- Missing values ---")
    print(data.isnull().sum())
    print("\n--- Duplicate rows ---")
    print(data.duplicated().sum())
    print("\n--- Invalid quantities (zero or negative) ---")
    print(data.loc[data["quantity"] <= 0, ["order_id", "quantity"]])
    print("\n--- Invalid prices (zero or negative) ---")
    print(data.loc[data["price"] <= 0, ["order_id", "price"]])

    converted_dates = get_converted_dates(data)
    invalid_dates = data["order_date"].notna() & converted_dates.isna()
    print("\n--- Invalid dates ---")
    print(data.loc[invalid_dates, ["order_id", "order_date"]])
```

1. Uses date conversion to safely identify invalid dates.

2. Checks missing values, duplicates, invalid quantities, prices, and dates.

Coforge

Data Validation and Quality Audit — Part 2

src/data_validator.py · Data-quality summary creation

src/data_validator.py

```python
def create_data_quality_summary(data):
    """Create a simple summary of data-quality checks and actions."""
    converted_dates = get_converted_dates(data)

    return pd.DataFrame(
        {
            "check": [
                "Duplicate rows", "Missing customer", "Missing city",
                "Missing or invalid date", "Missing price", "Invalid quantity",
                "Invalid price",
            ],
            "count": [
                data.duplicated().sum(), data["customer"].isnull().sum(),
                data["city"].isnull().sum(), converted_dates.isnull().sum(),
                data["price"].isnull().sum(), (data["quantity"] <= 0).sum(),
                (data["price"] <= 0).sum(),
            ],
            "action_taken": [
                "Removed", "Replaced with Unknown", "Replaced with Unknown",
                "Removed", "Removed", "Removed", "Removed",
            ],
```

1. Creates a quality-audit summary with counts and actions.
2. Counts duplicates, missing values, invalid dates, quantities and prices.
3. main.py saves this table as a CSV audit report in output/.

Coforge

Cleaning and Transforming Sales Data — Part 1

src/transformations.py · Duplicates, missing values and date conversion

src/transformations.py

```python
def clean_sales_data(data):
    """Clean the sales data before calculations are performed."""
    cleaned_data = data.drop_duplicates().copy() # exact duplicates are removed

    # rows become from 123 to 120 after removing duplicates

    cleaned_data["customer"] = cleaned_data["customer"].fillna("Unknown") #if name is missing it is replaced with "Unknown"

    cleaned_data["city"] = cleaned_data["city"].fillna("Unknown") #if city is missing it is replaced with "Unknown"

    cleaned_data["order_date"] = pd.to_datetime(
        cleaned_data["order_date"], errors="coerce", format="mixed"
    ) # convert order_date to datetime, invalid parsing will be set as NaT(coerce)
    # mixed for format because the data has multiple formats for order_date
```

1 Removes duplicates and fills missing customer/city values with Unknown.

2 Converts order_date to datetime; invalid dates become NaT.

Coforge

Cleaning and Transforming Sales Data — Part 2

src/transformations.py · Removing invalid records and creating sales_amount

src/transformations.py

```
    cleaned_data = cleaned_data.dropna(subset=["order_date", "price"])
    # removed rows with missing order_date or price
    # ( 2 missing or invalid dates and 1 missing price)
    # 120 becomes 117 after removing rows with missing order_date or price

    cleaned_data = cleaned_data[
        (cleaned_data["quantity"] > 0) & (cleaned_data["price"] > 0)
    ] # removes rows with negative quantity or price
    #( 1 negative quantity , 1 negative prices and 1 zero quantity)
    # 117 becomes 114 after removing rows with non-positive quantity or price

    return cleaned_data


def add_sales_amount(data):
    """Add sales_amount by multiplying quantity and price."""
    data = data.copy()
    data["sales_amount"] = data["quantity"] * data["price"]
    return data
```

1 Removes rows with a missing date or price, then non-positive quantity or price.

2 Creates sales_amount using quantity multiplied by price.

Cleaning result

123 Raw Records → 114 Clean Records

Coforge

Outlier Detection Using IQR

src/outlier_detection.py

src/outlier_detection.py

```python
def find_outliers(data):
    """Find unusual quantity or price values using the IQR method."""
    outlier_rows = []

    for column in ["quantity", "price"]:
        first_quartile = data[column].quantile(0.25)
        third_quartile = data[column].quantile(0.75)
        interquartile_range = third_quartile - first_quartile
        lower_limit = first_quartile - 1.5 * interquartile_range
        upper_limit = third_quartile + 1.5 * interquartile_range

        outliers = data[(data[column] < lower_limit) | (data[column] > upper_limit)].copy()
        outliers["outlier_column"] = column
        outliers["lower_limit"] = lower_limit
        outliers["upper_limit"] = upper_limit
        outlier_rows.append(outliers)

    return pd.concat(outlier_rows, ignore_index=True)
```

1. Calculates first quartile, third quartile, and interquartile range.
2. Uses lower and upper limits to identify unusual quantity or price values.
3. Exports flagged values to outlier_report.csv.

Result: No IQR-based quantity or price outliers were found.

Coforge

Creating Sales Reports

src/reports.py — create_reports()

src/reports.py

```python
def create_reports(data):
    """Create the grouped sales reports required by the project."""
    product_sales = data.groupby("product", as_index=False)["sales_amount"].sum()
    top_products = product_sales.sort_values("sales_amount", ascending=False).head(5)

    customer_sales = data.groupby("customer", as_index=False)["sales_amount"].sum()
    top_customers = customer_sales.sort_values("sales_amount", ascending=False).head(5)

    city_sales = data.groupby("city", as_index=False)["sales_amount"].sum()
    city_sales = city_sales.sort_values("sales_amount", ascending=False)

    category_sales = data.groupby("category", as_index=False)["sales_amount"].sum()
    category_sales = category_sales.sort_values("sales_amount", ascending=False)

    data = data.copy()
    data["month"] = data["order_date"].dt.strftime("%Y-%m")
    monthly_sales = data.groupby("month", as_index=False)["sales_amount"].sum()

    return top_products, top_customers, city_sales, category_sales, monthly_sales
```

1. Groups and sums sales by product, customer, city, and category.
2. Sorts products/customers from highest to lowest sales.
3. Creates a YYYY-MM month column for monthly sales analysis.

Coforge

Exporting Summary CSV Reports

src/reports.py — export_reports()

src/reports.py

```python
def export_reports(total_sales, average_order_value, top_products, top_customers,
                   city_sales, monthly_sales):
    """Save the required report tables as CSV files."""
    sales_summary = pd.DataFrame(
        {"total_sales": [round(total_sales, 2)],
         "average_order_value": [round(average_order_value, 2)]}
    )
    sales_summary.to_csv(f"{OUTPUT_FOLDER}/sales_summary.csv", index=False)
    top_customers.to_csv(f"{OUTPUT_FOLDER}/customer_summary.csv", index=False)
    top_products.to_csv(f"{OUTPUT_FOLDER}/product_summary.csv", index=False)
    city_sales.to_csv(f"{OUTPUT_FOLDER}/city_summary.csv", index=False)
    monthly_sales.to_csv(f"{OUTPUT_FOLDER}/monthly_sales.csv", index=False)
```

1. Creates sales_summary.csv containing total sales and average order value.
2. Exports customer, product, city, and monthly sales reports.
3. Saves reports in the output folder.

Coforge

Creating and Saving Sales Charts — Part 1

src/charts.py · City and category bar charts

src/charts.py

```python
def create_charts(city_sales, category_sales, monthly_sales):
    """Create, save, and display simple sales charts."""
    plt.figure(figsize=(8, 5))
    plt.bar(city_sales["city"], city_sales["sales_amount"], color="#002A5B")
    plt.title("Sales by City")
    plt.xlabel("City")
    plt.ylabel("Sales Amount")
    plt.xticks(rotation=30)
    plt.tight_layout()
    plt.savefig(f"{OUTPUT_FOLDER}/sales_by_city.png")
    print(f"Chart saved: {OUTPUT_FOLDER}/sales_by_city.png")

    plt.figure(figsize=(8, 5))
    plt.bar(category_sales["category"], category_sales["sales_amount"], color="#FF5C43")
    plt.title("Sales by Category")
    plt.xlabel("Category")
    plt.ylabel("Sales Amount")
    plt.tight_layout()
    plt.savefig(f"{OUTPUT_FOLDER}/sales_by_category.png")
    print(f"Chart saved: {OUTPUT_FOLDER}/sales_by_category.png")
```

1. Creates bar charts for city and category sales.
2. Uses the project navy and coral colours.
3. Saves each PNG chart file in output/.

Coforge

Creating and Saving Sales Charts — Part 2

src/charts.py · Monthly sales line chart

src/charts.py

```
    plt.figure(figsize=(8, 5))
    plt.plot(monthly_sales["month"], monthly_sales["sales_amount"], marker="o", color="#FF5C43")
    plt.title("Monthly Sales")
    plt.xlabel("Month")
    plt.ylabel("Sales Amount")
    plt.grid(color="#D9E2EC")
    plt.tight_layout()
    plt.savefig(f"{OUTPUT_FOLDER}/monthly_sales_chart.png")
    print(f"Chart saved: {OUTPUT_FOLDER}/monthly_sales_chart.png")

    plt.show()
```

1 Creates a monthly sales line chart.

2 Saves PNG chart files in output/ and displays them.

Application Logging

src/logger_setup.py

src/logger_setup.py

```python
def setup_logger():
    """Create and return the project logger."""
    logs_folder = Path(LOG_FOLDER)
    logs_folder.mkdir(exist_ok=True)

    logging.basicConfig(
        filename=logs_folder / "app.log",
        level=logging.INFO,
        format="%(asctime)s - %(levelname)s - %(message)s",
        force=True,
    )
    return logging.getLogger(__name__)
```

1. Creates the logs folder if it does not exist.
2. Saves execution information in logs/app.log.
3. Records project progress, warnings, and errors.

Coforge

Conclusion

Cleaned 123 raw records into 114 valid records

Generated accurate sales insights and exportable reports

Added charts, logging, audit reporting, outlier review, and automated tests

The project demonstrates a simple, reliable, and explainable Python + Pandas data-analysis workflow.

Thank You , Any Questions?

Coforge

Outputs, Logging, and Data Quality Audit

output/ — 7 CSV reports
sales_summary.csv
customer_summary.csv
product_summary.csv
city_summary.csv
monthly_sales.csv
data_quality_summary.csv
outlier_report.csv

Logging

logs/app.log records important processing steps.

Outlier check

No IQR-based quantity or price outliers were found in the clean dataset.

Project Execution Output

Complete workflow executed using: py main.py

```
Clean rows ready for analysis: 114
Outlier report saved: output/outlier_report.csv
Total sales: 229905.0
Average order value: 2016.71
```

114
Clean Records Ready for Analysis

7
CSV Reports Generated

3
PNG Charts Generated

The complete workflow successfully cleans data, calculates sales insights, exports reports, and generates charts in one run.

```
PS D:\DE_FinalProject\sales_analysis> py main.py
4     Mumbai       47160.0
2      Delhi       25660.0
3  Hyderabad       23400.0
1    Chennai       22480.0
6    Unknown        1400.0

--- Sales by category ---
      category  sales_amount
2         Home      114180.0
1  Electronics       63100.0
0  Accessories       41150.0
3   Stationery       11475.0

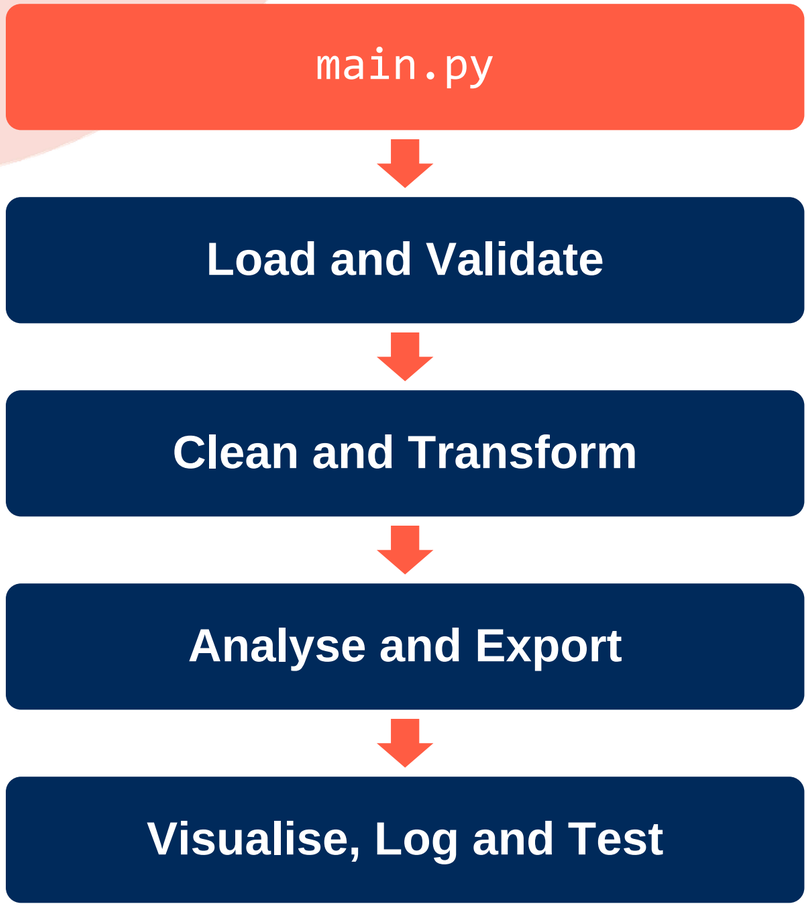
--- Monthly sales ---
     month  sales_amount
0  2025-01       33510.0
1  2025-02       33860.0
2  2025-03       38135.0
3  2025-04       46000.0
4  2025-05       40685.0
5  2025-06       19465.0
6  2025-07       18250.0

CSV report files saved in the output folder.
Chart saved: output/sales_by_city.png
Chart saved: output/sales_by_category.png
Chart saved: output/monthly_sales_chart.png
```


Coforge

Implementation Code Walkthrough

High-resolution explanation of the actual Python modules



`main.py`	Entry point that runs every step in order
`data_loader.py · data_validator.py`	Reads sales.csv and audits data quality
`transformations.py · outlier_detection.py`	Cleans records, adds sales_amount, checks outliers
`reports.py`	Groups sales and exports CSV reports
`charts.py · logger_setup.py · tests/`	Saves charts, records logs, runs Pytest checks

Coforge